

KIẾN TRÚC CHO CRYPTO STRATEGY LAB

Từ "chạy được" đến một nền tảng có thể thay đổi, mở rộng và kiểm chứng

Software Architecture

Mục tiêu: thấy một yêu cầu thay đổi làm hệ thống "vỡ" — rồi dùng kiến thức kiến trúc để cứu nó.

Đối chiếu syllabus: Topic 1 → 11 · **Cơ sở:** [S], [P], [R1–R26], [W1–W8]

Tran Quy

0. Bố cục

Phần	Thời gian	Vấn đề trọng tâm	Syllabus
Mở màn	0–5'	Bot trade đầu tiên	Topic 1
Act 1	5–15'	Yêu cầu mới làm code vỡ	Topic 2
Act 2	15–27'	Không có bản đồ chung	Topic 3
Act 3	27–42'	God Service và coupling	Topic 4, 6
Act 4	42–58'	100.000 strategy candidates	Topic 8, 9
Act 5	58–72'	Scale, deploy, failure	Topic 5
Act 6	72–84'	Kết quả có tái lập được không?	Topic 8, 10, 11
Act 7	84–97'	Kiến trúc đẹp có thật sự tốt?	Topic 4 – ADD/ATAM
Kết	97–100'	Architecture Proof	Review

1. Nhiệm vụ: Crypto Strategy Lab cần làm gì?

- Market data realtime từ Binance (nhiều timeframe) · nhiều strategy (MA, RSI, Bollinger...)
- Composite strategy, backtest, evaluate + leaderboard, tự động search combinations
- News → Sentiment ML · mở rộng được **không cần viết lại toàn bộ hệ thống**

Trọng tâm là Kiến trúc phần mềm, không phải tìm strategy đầu tư tốt nhất.

Nguồn: [P §2, §47] · [S Topic 1]

2. Bot đầu tiên: "Chạy rồi!"

Mọi thứ được viết vào **một class duy nhất**:

```
TradingService: getBinanceData, calculateMA, calculateRSI, crawlNews,  
analyzeSentiment, backtest, rank, saveDatabase, sendWebSocket
```

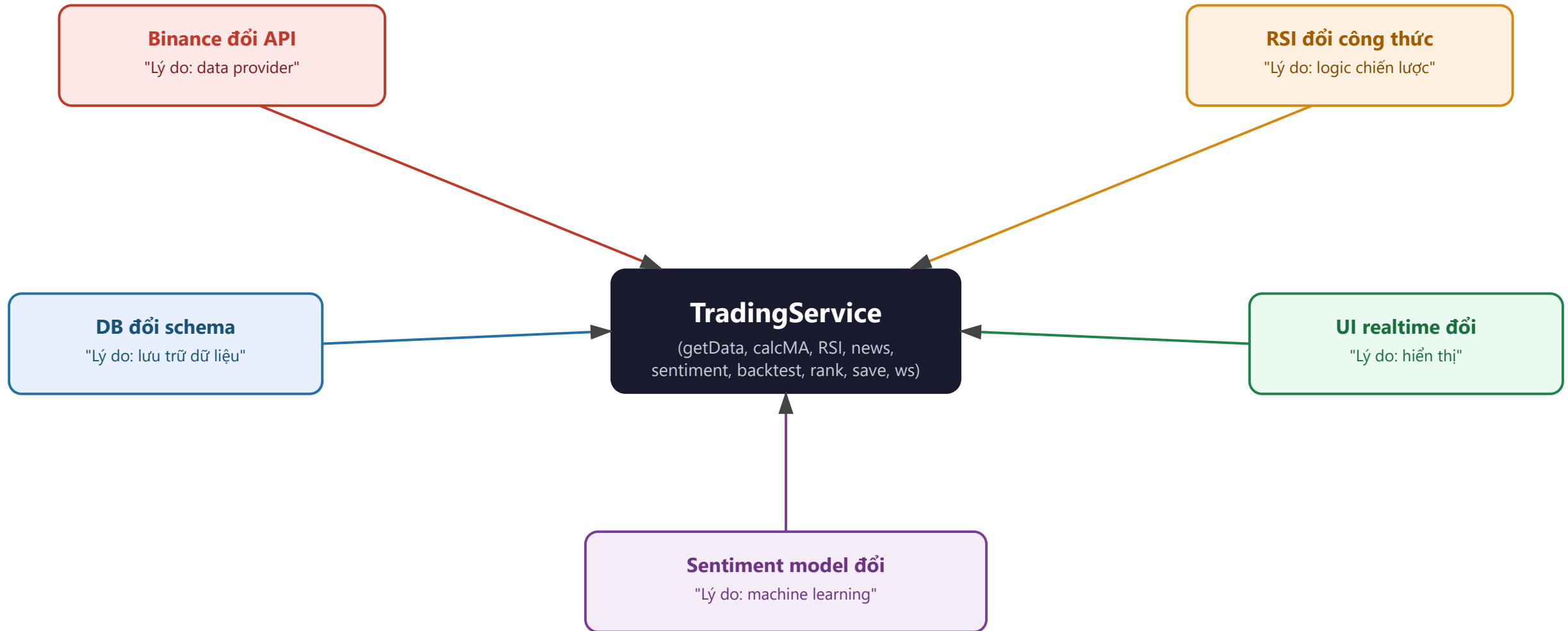
Có thể chạy rất tốt. Đây gọi là "God Service" — một class biết và làm mọi thứ.

Chưa sai về chức năng — nhưng **nguy hiểm khi hệ thống phải thay đổi**, vì mọi lý do thay đổi trên đời đều dồn vào đúng một chỗ.

Syllabus: Topic 1 – Software Architecture Concepts

Nguồn: [P §44 – God Service anti-pattern] · [R2] · [R10]

God Service: 5 lý do thay đổi, 1 nơi bị đụng



5 lý do thay đổi khác nhau — nhưng chỉ có **MỘT** chỗ trong code phải sửa mỗi lần.

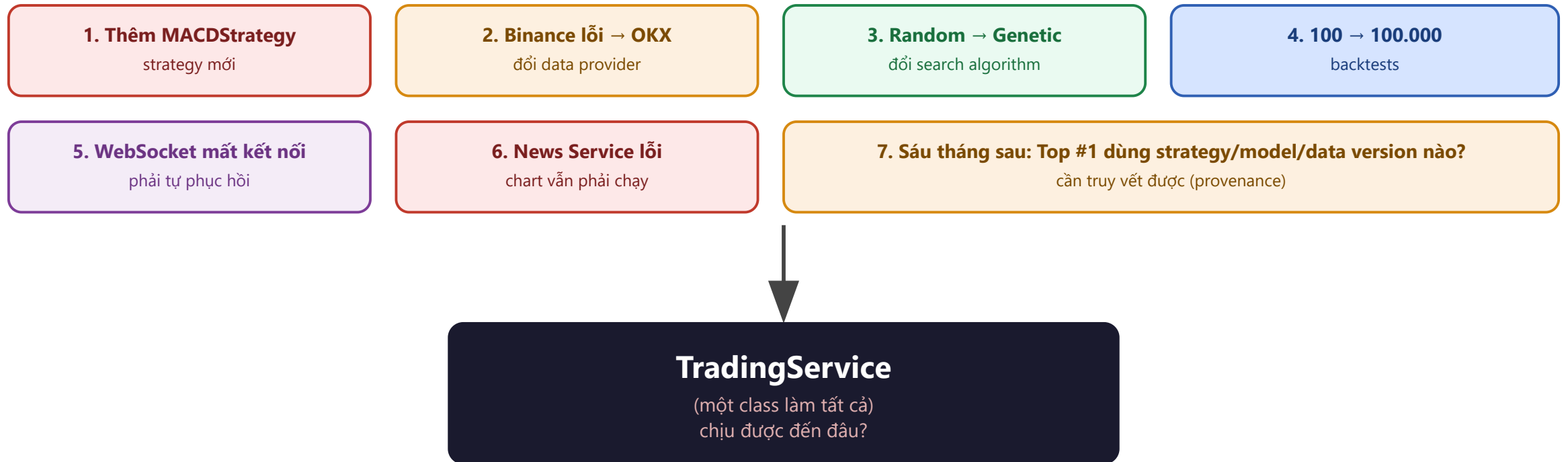
3. Bảy yêu cầu thay đổi áp đến

Mỗi thay đổi buộc ta sửa **bao nhiêu** nơi?

(sơ đồ đầy đủ ở slide sau)

Nguồn: [P §40–43]

7 yêu cầu thay đổi ập đến cùng lúc



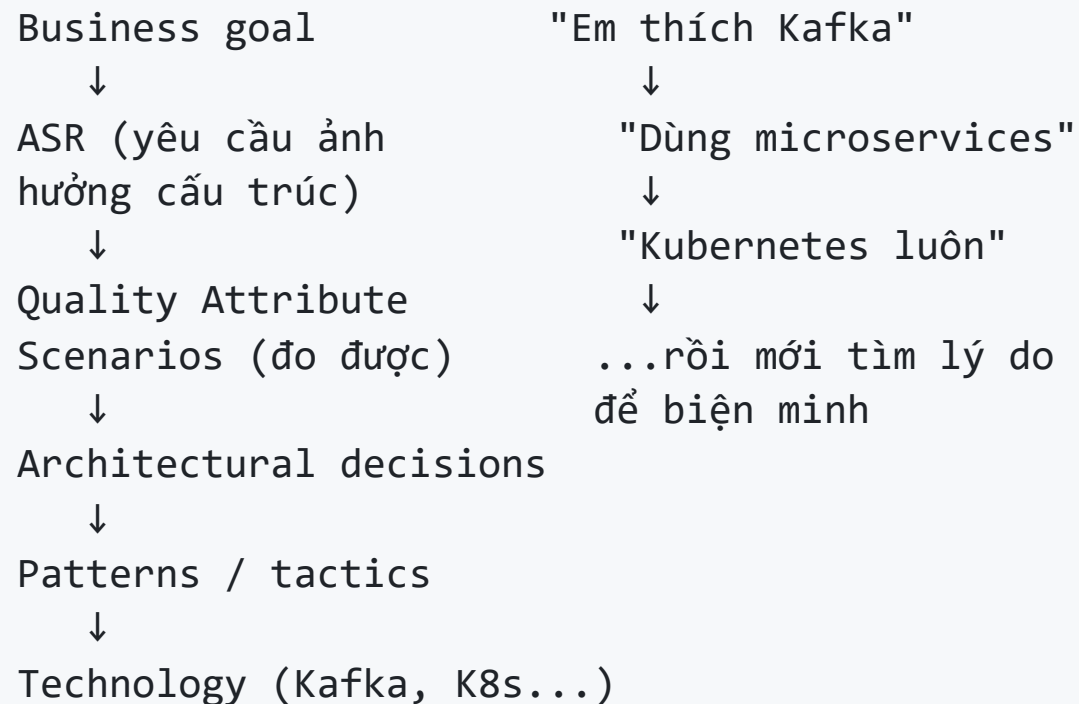
Mỗi thay đổi buộc sửa BAO NHIÊU nơi?

Đây là câu hỏi mà kiến trúc phần mềm phải trả lời được — không phải "chạy có được không".

ACT 1 — Không bắt đầu bằng Kafka

Bắt đầu bằng **Architectural Drivers** (lý do buộc ta phải thiết kế theo cách nhất định)

Đúng: đi từ lý do đến công cụ · Sai: chọn công cụ trước rồi mới tìm lý do



Syllabus: Topic 2 – ASRs, Quality Attributes

Nguồn: [R1], [R6], [W1]

4. Architectural Drivers của Crypto Strategy Lab

8 "đèn cảnh báo" mà kiến trúc phải thiết kế sẵn (*xem bảng điều khiển ở slide sau*)

Nguồn: [P §32, §36] · Syllabus Topic 2

8 "đèn cảnh báo" của Crypto Strategy Lab

"Xe chạy tốt" chưa đủ — mỗi đèn dưới đây phải sáng đúng lúc

Modifiability

Thêm MACD phải
sửa mấy nơi?

Scalability

100 → 100.000
candidates thì sao?

Performance

1.000 backtests: tuần tự
hay song song?

Realtime

Candle mới đến chart
trễ bao lâu?

Reliability

Binance disconnect
có mất candle?

Maintainability

Đổi search algorithm có
viết lại backtester?

Observability

Loop đang chạy?
Bao nhiêu job lỗi?

Reproducibility

Top #1 sinh ra từ
version nào?

Mỗi driver cần một Quality Attribute Scenario

đo được, kiểm tra được — không phải slogan mơ hồ
như "nhanh", "ổn định", "dễ mở rộng"

Không có driver nào quan trọng hơn driver nào — quan trọng là driver nào ĐÚNG cho bài toán này.

5. Biến "hệ thống phải tốt" thành scenario đo được

Template đơn giản (nhớ tắt: S-S-E-A-R-M)

SOURCE	Ai/cái gì gây sự kiện?
STIMULUS	Chuyện gì xảy ra?
ENVIRONMENT	Trong hoàn cảnh nào?
ARTIFACT	Thành phần nào bị tác động?
RESPONSE	Hệ thống phải làm gì?
MEASURE	Đo thế nào để biết đạt?

Modifiability: thêm MACDStrategy → đăng ký plugin mới
→ không sửa Backtester, Evaluator, UI

Reliability: Binance WebSocket disconnect → reconnect
+ recover missing candles → không mất/duplicate

Nguồn: [R1], [R6]

6. Mini challenge: "Yêu cầu nào mới thật sự lái kiến trúc?"

A

"Chart phải đẹp."

B

"Khi đổi timeframe 5m → 1h, chỉ chart đó cập nhật, không reload toàn hệ thống."

C

"Logo đặt góc trái."

D

"News Service lỗi không làm Market Data pipeline ngừng."

Chọn ASR (yêu cầu ảnh hưởng cấu trúc) và giải thích vì sao

Đáp án kỳ vọng: B, D — vì cả hai đều buộc hệ thống phải có ranh giới (boundary) và cơ chế cách ly (isolation) rõ ràng, còn A và C chỉ là chi tiết giao diện, không đòi hỏi thay đổi cấu trúc.

ACT 2 — "Mỗi người đang hình dung một hệ thống khác nhau"

Ta cần **bản đồ**

Architecture documentation không phải để làm đẹp báo cáo.

Nó là **shared mental model** — bản đồ chung để cả nhóm nhìn về một hướng.

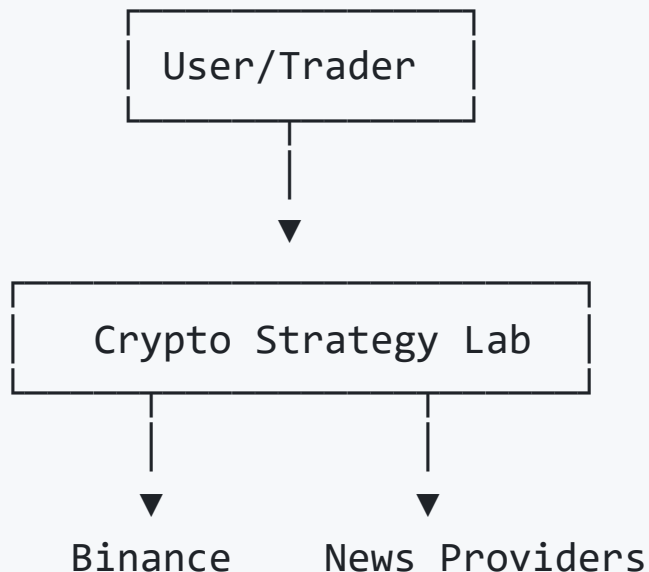
Syllabus Topic 3

- 4+1 View Model (5 góc nhìn kiến trúc)
- C4 Model (4 mức phóng to dần)
- Architecture Views (góc nhìn kiến trúc)
- UML Architecture Diagrams

Nguồn: [R5], [R7], [R8], [W2]

7. C4 Level 1 — System Context

"Hệ thống của chúng ta sống trong thế giới nào?"



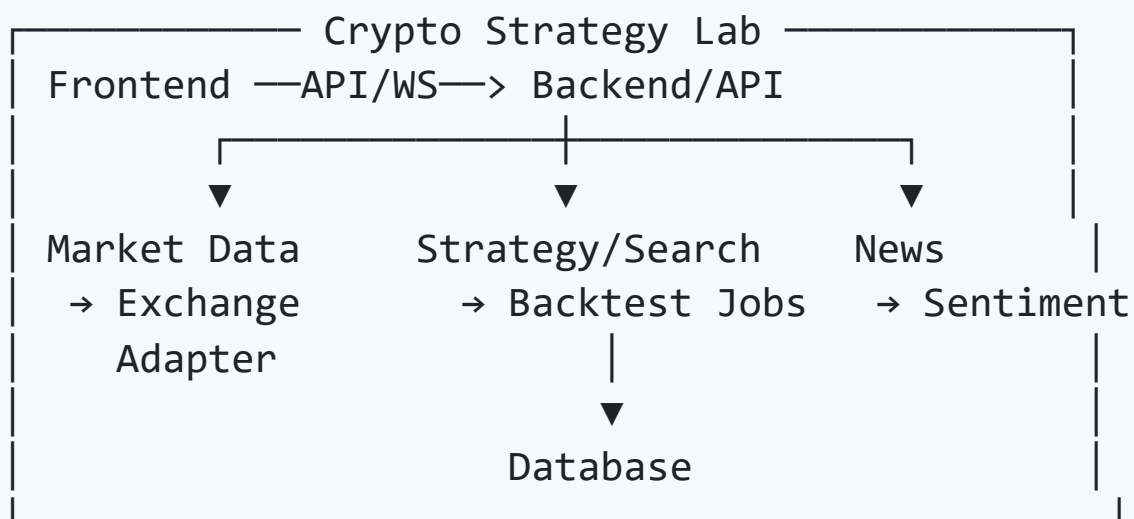
Không cần đưa vào đây

Redis · Kafka · React · PostgreSQL · Python

Mục đích: boundary + people + external systems — trả lời câu "hệ thống này nói chuyện với ai ở bên ngoài?", chưa quan tâm công nghệ bên trong.

8. C4 Level 2 — Container

"Bên trong Crypto Strategy Lab có những khối chạy độc lập nào?"



Candidate architecture — một đề xuất khả dĩ, không phải đáp án duy nhất.

Project: [P §31]

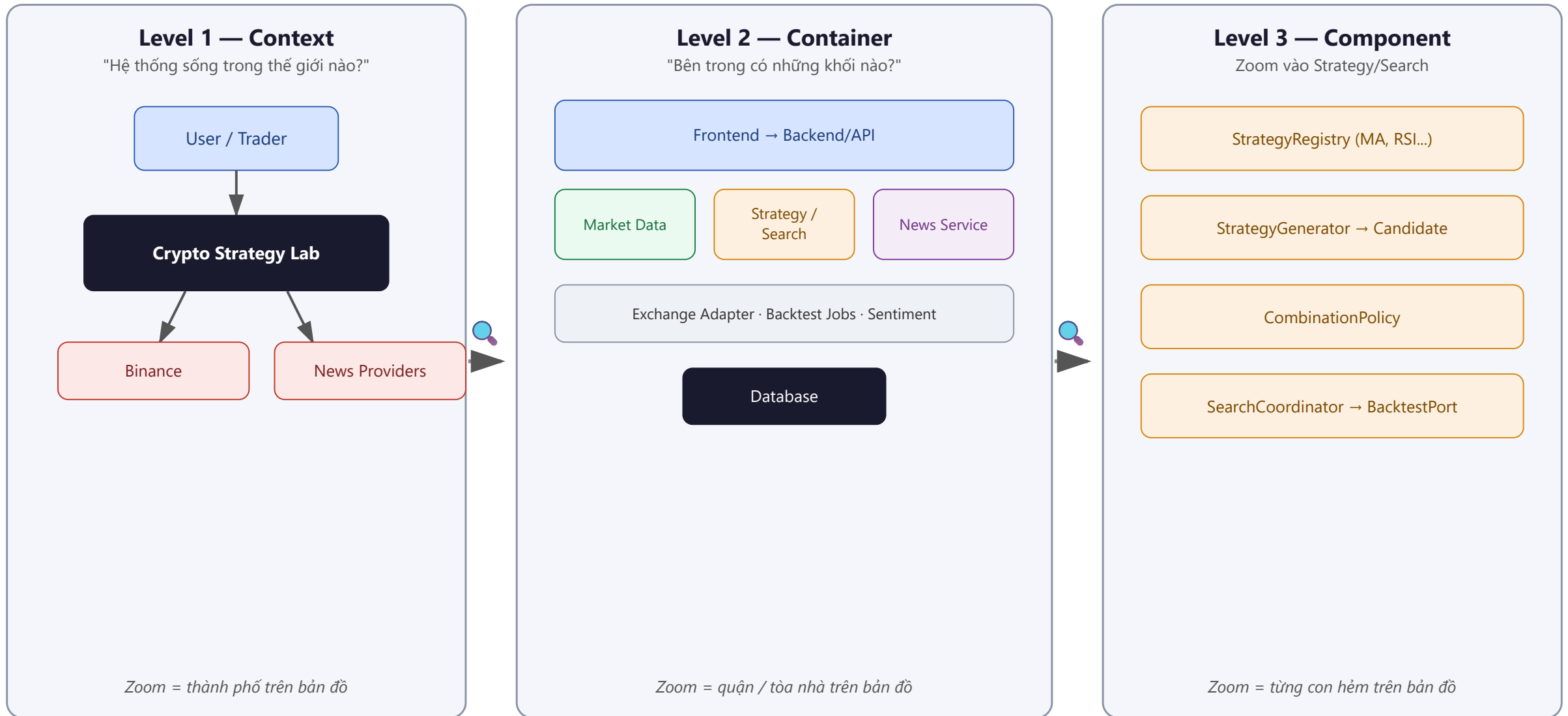
9. C4 Level 3 — Component

Zoom vào Strategy/Search

Strategy/Search: StrategyRegistry (MA, RSI...) · StrategyGenerator → CandidateStrategy
→ CombinationPolicy · SearchCoordinator → BacktestPort

Câu hỏi view này trả lời: Ai đăng ký strategy? Ai sinh candidate? Ai combine signal? Search có biết backtest implementation không? *(sơ đồ 3 mức ở slide sau)*

C4 Model: cùng một hệ thống, 3 mức phóng to



Cùng một hệ thống — chỉ khác câu hỏi đang muốn trả lời và mức độ chi tiết cần thiết.

10. Static view chưa đủ: hãy kể một runtime story

Scenario: candidate mới lên Leaderboard ("dynamic view" — góc nhìn theo thời gian)

```
User → START SEARCH → SearchCoordinator → generate → CandidateStrategy  
    → enqueue → BacktestWorker → result → Evaluator → score  
    → Ranking → LeaderboardUpdated → Frontend
```

Cho thấy điều sơ đồ tĩnh không thể hiện được:

sync/async boundary · data flow · failure point · latency path

Đối chiếu: 4+1 / Architecture Views · [R5], [R7]

ACT 3 — Chia đúng trách nhiệm trước khi chia server

"Một nhà hàng không để đầu bếp kiêm thu ngân, shipper và kế toán"

Chef	→ nấu
Cashier	→ thanh toán
Waiter	→ phục vụ
Accountant	→ kế toán

Crypto Lab cũng vậy — mỗi khối chỉ nên có một lý do để thay đổi:

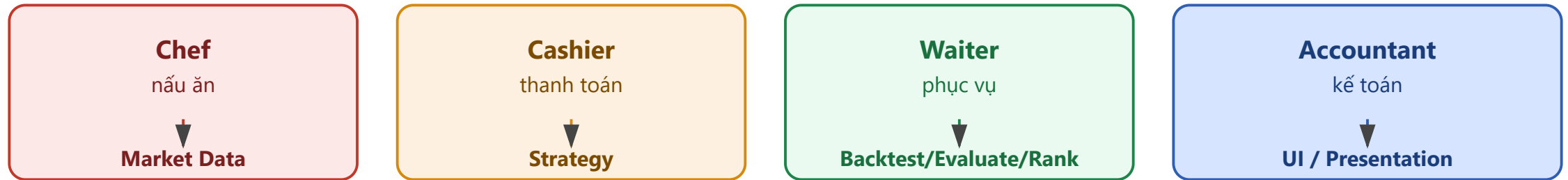
MarketData ≠ Strategy ≠ Backtest ≠ Evaluate ≠ Rank ≠ UI

Syllabus: Topic 4 + Topic 6

Nguồn: [R2], [R3], [R9], [R10], [R20], [R21]

(sơ đồ ánh xạ đầy đủ ở slide sau)

Mỗi vai trò một trách nhiệm — một lý do để thay đổi



Mỗi khối chỉ nên có **MỘT** lý do để thay đổi

MarketData $\neq$ Strategy $\neq$ Backtest $\neq$ Evaluate $\neq$ Rank $\neq$ UI
6 slide tiếp theo sẽ bóc tách từng ranh giới này

*Nếu đầu bếp kiêm luôn thu ngân: đổi máy tính tiền cũng phải nghỉ nấu để học lại —
dù món ăn chẳng liên quan gì đến máy tính tiền.*

11. DDD: Đừng để "Trading" trở thành một khối mơ hồ

Có thể nhìn domain thành các capability (Domain-Driven Design — thiết kế theo nghiệp vụ)

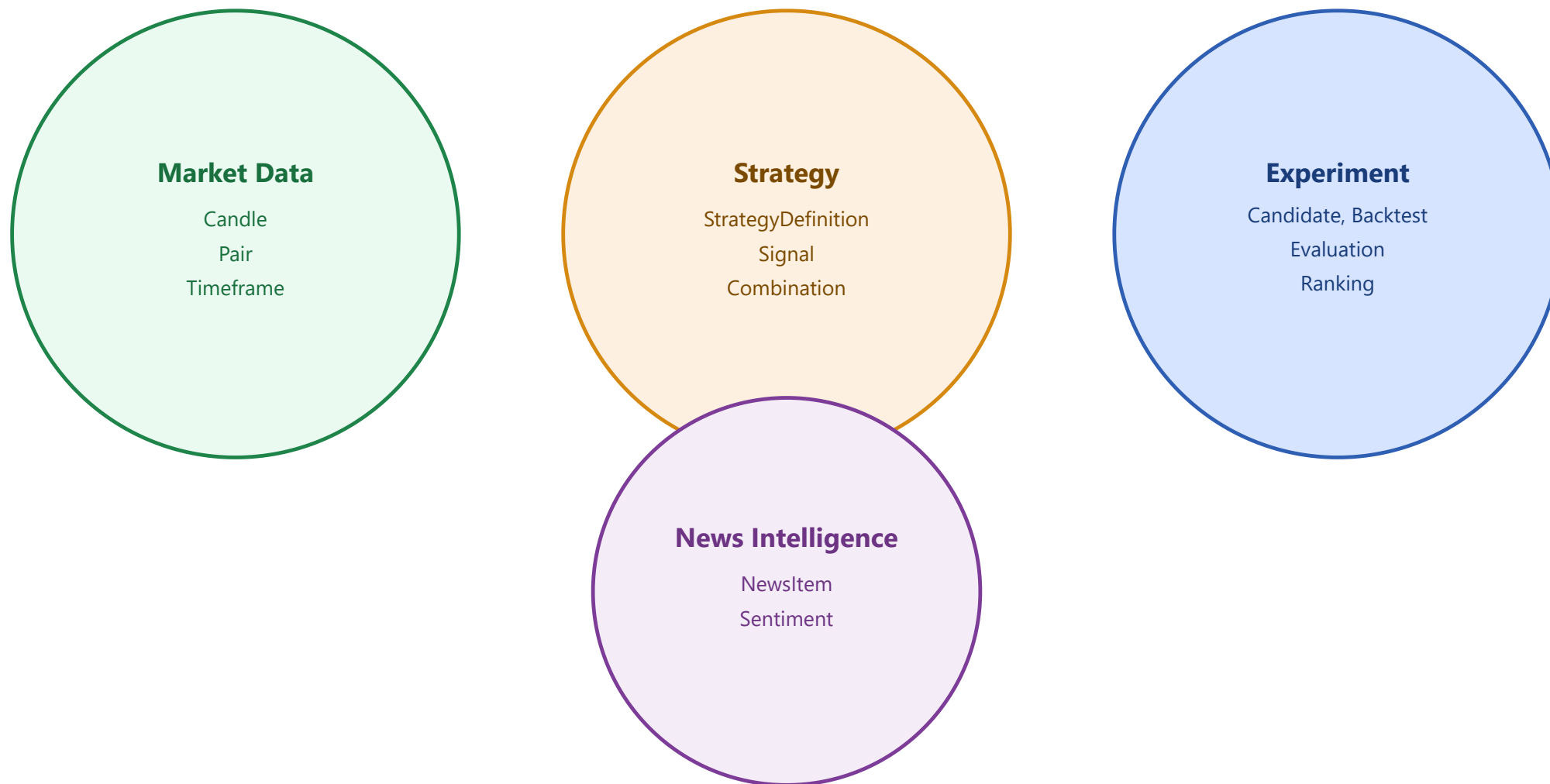
Bounded Context	Khái niệm bên trong
Market Data	Candle, Pair, Timeframe
Strategy	StrategyDefinition, Signal, Combination
Experiment	Candidate, Backtest, Evaluation, Ranking
News Intelligence	NewsItem, Sentiment

`Signal` trong Strategy Context không nhất thiết là cùng khái niệm với `Trade` trong Experiment Context.

Syllabus: Domain-Driven Design

Nguồn: [R3], [R20], [R21]

4 Bounded Context — 4 "thế giới" khái niệm riêng biệt



*`Signal` ở Strategy Context $\neq$ `Trade` ở Experiment Context —
cùng nói về "mua/bán" nhưng khác nghĩa, giống từ "Ly" đổi nghĩa theo ngữ cảnh.*

12. Clean Architecture: dependency hướng vào policy

Không nên: `RSIStrategy` → `MySQL` → `Binance JSON` (business logic tự gọi hạ tầng)

Nên: `RSIStrategy` → `MarketContext/Port` → `BinanceAdapter` | `RepositoryAdapter`
(đảo hướng phụ thuộc – dependency inversion)

Strategy cần data nó cần, không cần biết data đến từ đâu. (Sơ đồ đầy đủ ở slide sau.)

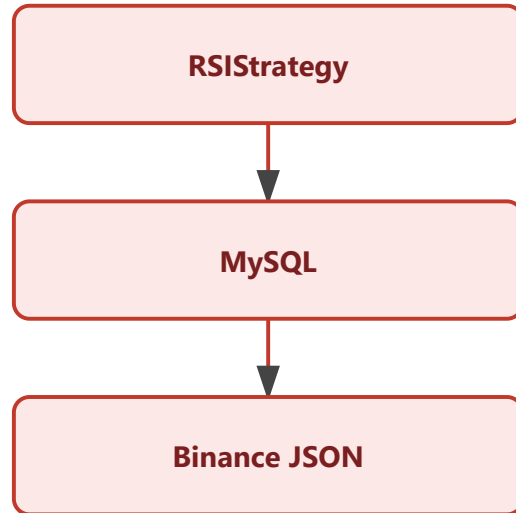
Project: [P §44]

Syllabus: The Clean Architecture

Nguồn: [R2]

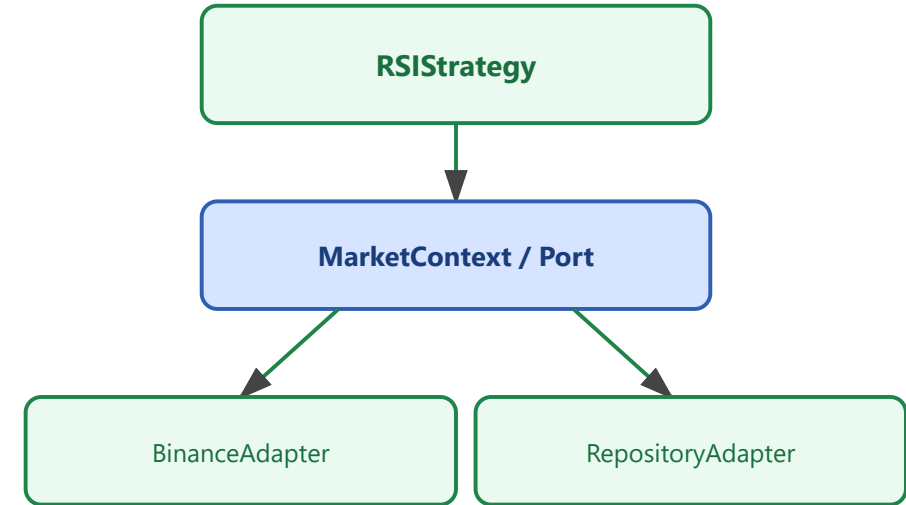
Clean Architecture: đảo hướng phụ thuộc qua "Port"

✗ Không nên



Business logic gọi thẳng
hạ tầng cụ thể → đổi DB/exchange
là phải sửa RSIStrategy.

✓ Nên



Strategy chỉ biết "Port" —
đổi database hay đổi sàn giao dịch
chỉ cần đổi Adapter, không đụng Strategy.

"Business policy không nên biết infrastructure cụ thể — chỉ nên biết một cổng giao tiếp chuẩn."

13. Strategy + Plugin + Registry

"Ngày mai có MACD thì sao?"

```
interface Strategy { analyze(context) -> Signal }
```

StrategyRegistry: MA, RSI, Bollinger, SupportResistance, + MACD ← thêm mới

Architecture test: thêm **MACDStrategy** mà **không sửa** Backtester, Evaluator, Leaderboard, Frontend core

Project: [P §12, §41]

Syllabus: Architectural Patterns

Nguồn: [R9]

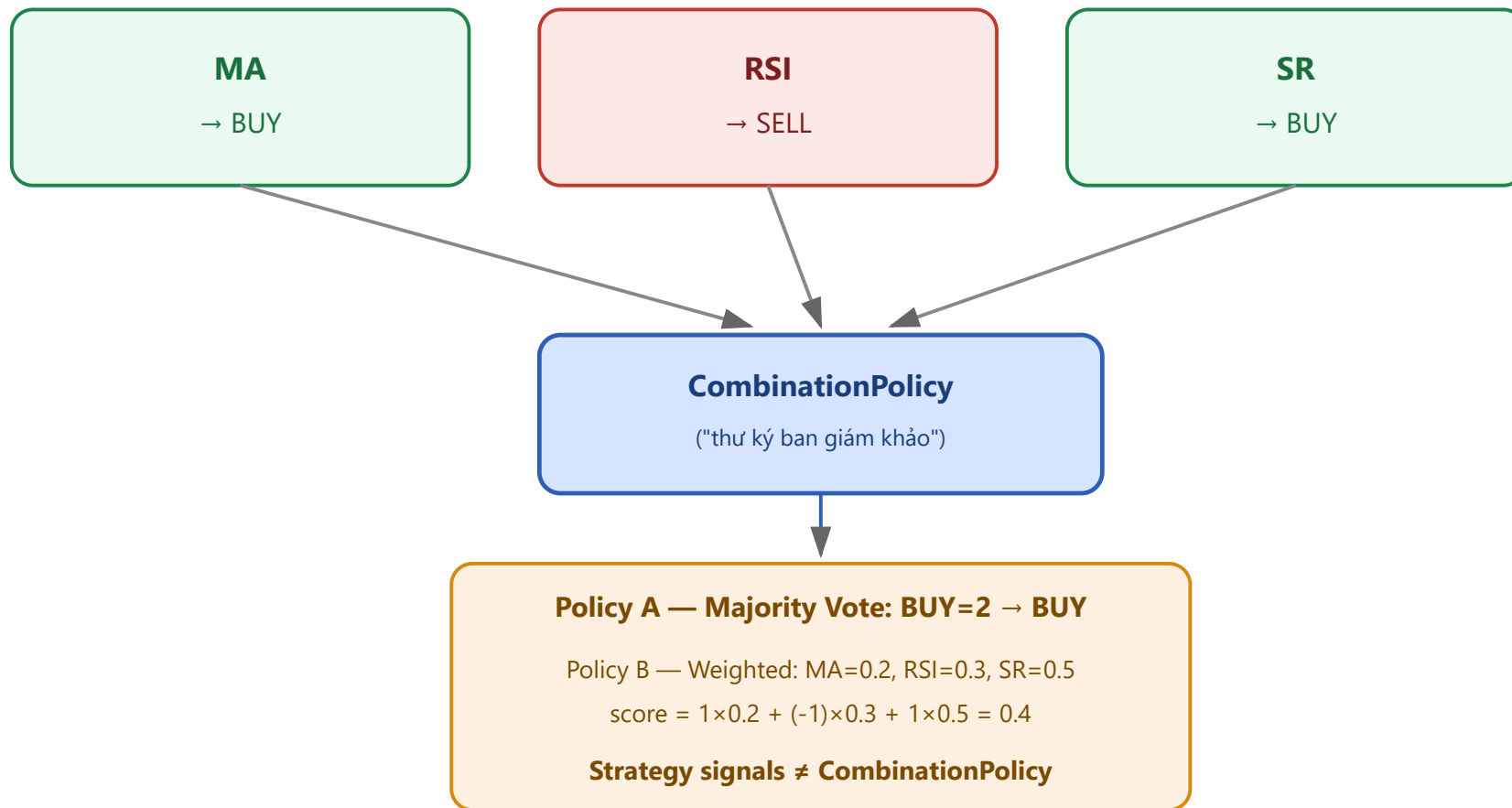
14. Composite Strategy: khi ba strategy "cãi nhau"

MA → BUY, RSI → SELL, SR → BUY – ai quyết định cuối cùng?

- Policy A — Majority Vote: BUY=2 → BUY
- Policy B — Weighted Vote: $\text{score} = 1 \times 0.2 + (-1) \times 0.3 + 1 \times 0.5 = 0.4$
- Kiến trúc cần tách: Strategy signals $\neq$ CombinationPolicy (*sơ đồ ở slide sau*)

Project: [P §13–14]

Khi 3 strategy "cãi nhau" — ai tổng hợp?



Mỗi giám khảo chỉ chấm điểm — không ai kiểm luôn việc tổng hợp.

15. Adapter: Đừng để frontend "nói tiếng Binance"

Sai: Frontend → Binance JSON (trực tiếp)

Tốt hơn: Frontend → MarketDataService → MarketDataProvider
→ BinanceAdapter | OKXAdapter | BybitAdapter

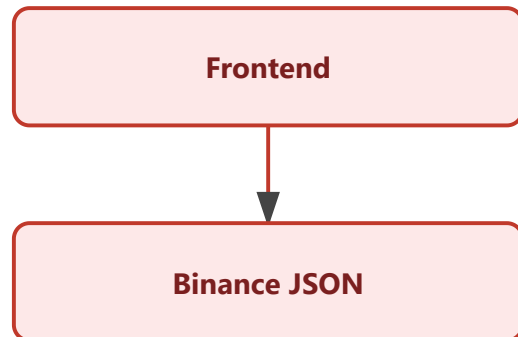
Normalized contract: `Candle { symbol, timeframe, openTime, open, high, low, close, volume }` (Sơ đồ đầy đủ ở slide sau)

Project: [P §4]

Nguồn: [R9], [R10]

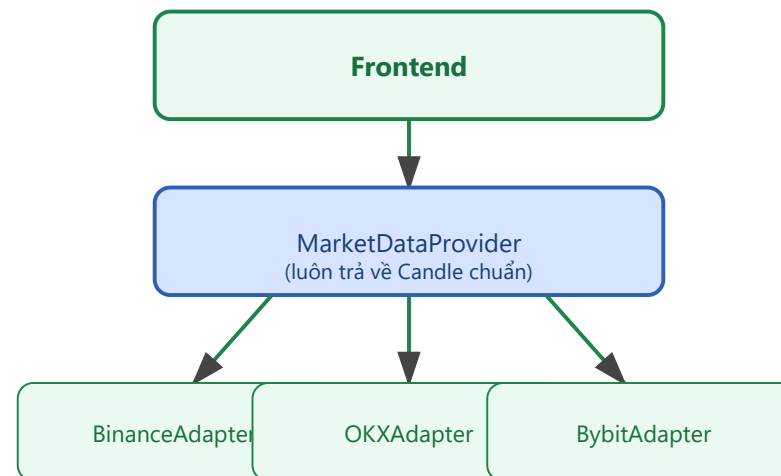
Adapter: frontend chỉ nên nói MỘT ngôn ngữ dữ liệu

✗ Sai



Frontend "học tiếng Binance".
Thêm OKX = frontend phải học
thêm một ngôn ngữ mới.

✓ Tốt hơn



Mỗi Adapter tự "phiên dịch" —
frontend chỉ cần biết đúng 1 định dạng Candle,
dù thêm bao nhiêu sàn mới.

Candle { symbol, timeframe, openTime, open, high, low, close, volume }

16. Frontend: SPA, Micro-Frontends, JAMstack — dùng khi nào?

- **MVP** hợp lý: Single SPA Dashboard (Chart, Strategy, Backtest, Leaderboard, News panel)
- **Micro-Frontend** chỉ đáng cân nhắc khi: nhiều team sở hữu feature độc lập, release cadence khác nhau, boundary UI rõ
- **JAMstack** hợp nội dung tĩnh/pre-render — realtime dashboard vẫn cần kênh dữ liệu thời gian thực riêng

(so sánh trực quan ở slide sau)

Syllabus Topic 6

Nguồn: [R12], [R16], [R17], [R18], [R19]

Frontend: dùng đúng style theo đúng bối cảnh

SPA — 1 bếp

Chart · Strategy · Backtest
Leaderboard · News

✓ MVP hợp lý

quán nhỏ, 1 đội quản lý
toàn bộ giao diện

Micro-Frontend

chuỗi nhượng quyền —
mỗi chi nhánh tự quản bếp

⚠ Chỉ khi cần

nhiều team độc lập,
release cadence khác nhau

JAMstack

tối ưu nội dung
tĩnh / pre-render

✗ Không hợp realtime

dashboard cần kênh dữ liệu
thời gian thực riêng

**Một nhóm nhỏ dùng mô hình nhượng quyền cho quán ăn nhỏ
là tự làm khó mình.**

Không có style nào được điểm vì tên nghe hiện đại — mỗi style có bối cảnh riêng.

17. Transaction boundary: một kết quả backtest "hoàn tất" nghĩa là gì?

Worker: 1. Lưu trades → 2. Lưu metrics → 3. Đánh dấu `COMPLETED` → 4. Publish `BacktestCompleted`

Nếu crash sau bước 2: Trades ✓, Metrics ✓, nhưng Status ✗, Event ✗ (sơ đồ ATM ở slide sau)

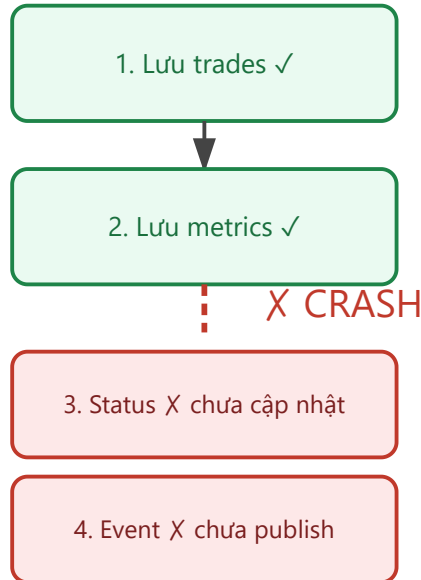
Câu hỏi kiến trúc: Atomicity cần tới đâu? Retry có tạo duplicate? Event publish và DB commit phối hợp thế nào?

Syllabus: Transactional Processing

Nguồn: [R11], [R10]

Giống chuyển tiền ATM: hoặc cả hai bước, hoặc không bước nào

✗ Nếu crash giữa chừng



**Leaderboard đọc dữ liệu
không nhất quán**

✓ Tư duy ATM: **MỘT** đơn vị

**Rút tiền ATM: trừ tài khoản A
+ cộng tài khoản B**

→ HOẶC cả hai cùng thành công,
→ HOẶC cùng bị huỷ (rollback)

Backtest worker cần tư duy y hệt:

Trades + Metrics + Status + Event
= một đơn vị toàn vẹn duy nhất

Câu hỏi kiến trúc:

Atomicity cần tới đâu? · Retry có tạo duplicate?
Event publish và DB commit phối hợp thế nào?

ACT 4 — Strategy Search làm hệ thống "nổ"

4 strategy thì vui. 4 strategy × nhiều parameters thì sao?

Generate → Backtest → Evaluate → Rank

Mỗi tham số nhân thêm (MA windows, RSI thresholds, BB deviations...) → bùng nổ tổ hợp. (*chi tiết ở slide sau*)

Project: [P §15–18]

Bùng nổ tổ hợp: vài tham số nhỏ → hàng chục nghìn candidate

Ví dụ: chọn trang phục



Với strategy trading

MA: 10/20, 20/50, 50/200
RSI: 14/30/70, 14/20/80...
BB: nhiều windows/deviations
SR: nhiều detection params
→ **hàng chục nghìn tổ hợp**



Generate → Backtest → Evaluate → Rank

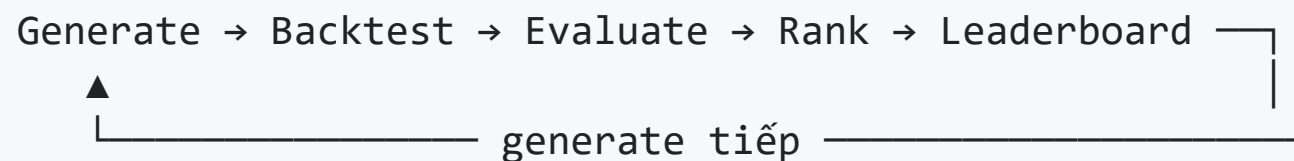
phải chạy được với khối lượng này

100.000 candidates

không còn là con số ngẫu nhiên — mà là hệ quả tất yếu của phép nhân tổ hợp.

18. Continuous Strategy Loop

"Generate → Execute → Measure → Improve"



Bắt buộc có Stop Condition (điều kiện dừng) — **không** `while(true)`

max candidates · max time · no improvement N iterations · user cancel

Project: [P §23–24]

19. Từ for-loop sang Job Queue + Workers

Kém scalable — một người làm hết

Kém scalable: `for candidate in candidates: backtest(); evaluate(); update_ui()`

Tách pipeline: `StrategyGenerator → Job Queue → [W1, W2, W3] → Evaluator → Ranking`

Mua được: parallelism · retry · pause/resume · backpressure · observability

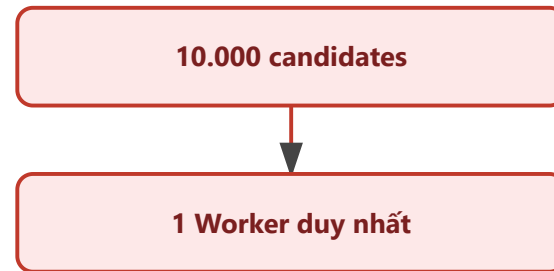
Project: [P §24, §43]

Syllabus: Message Brokers / EDA

Nguồn: [R4], [R13], [R22], [W3]

Một quầy thu ngân so với nhiều quầy song song

✗ For-loop tuần tự

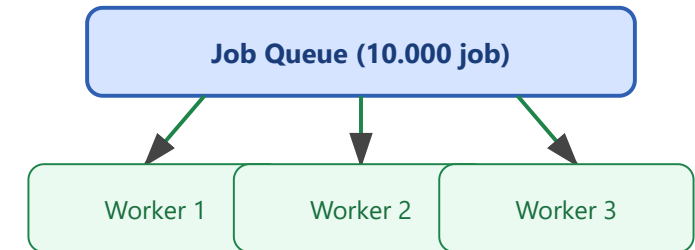


candidate #1 → #2 → #3 → ...
lần lượt, không cái nào xen ngang

≈ 5.5 giờ

(10.000 × 2 giây, chạy 1 luồng)

✓ Job Queue + Workers



nhiều worker xử lý song song,
thêm worker khi cần nhanh hơn

≈ 1.8 giờ (3 workers)

càng nhiều worker càng nhanh

Số liệu minh họa (1 worker ≈ 2 giây/candidate) — mục đích là cảm nhận độ lớn, không phải cam kết hiệu năng.

20. Event-Driven: "Tôi thông báo sự thật đã xảy ra"

Thay vì gọi chặt: `BacktestWorker` → direct call → `LeaderboardService.update()`

Publish event: `BacktestWorker` → `StrategyEvaluated` → `[Ranking, Audit]`

Producer không cần biết consumer là ai (*sơ đồ ở slide sau*)

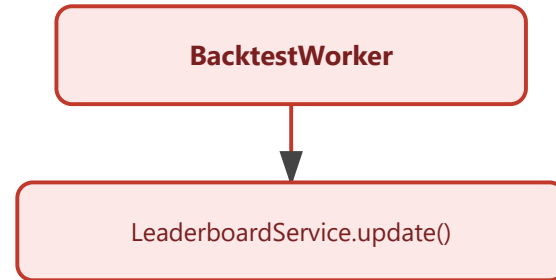
Project: [P §34]

Syllabus: Event-Driven Architecture

Nguồn: [R4], [R22], [W3]

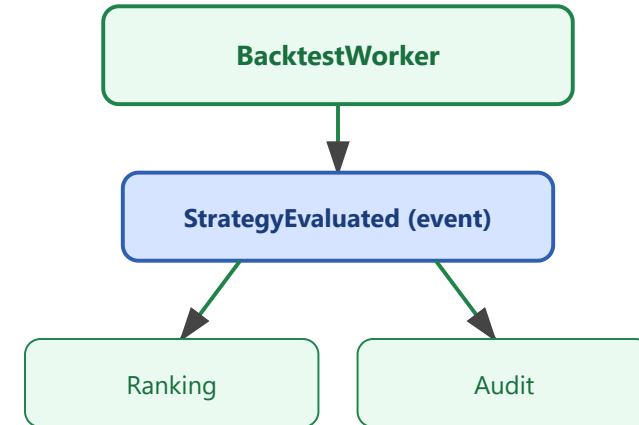
Gọi điện riêng từng người, so với phát loa thông báo

✗ Direct call (gọi chặt)



Thêm consumer mới (vd: Audit)
= phải sửa code BacktestWorker
để gọi thêm một hàm nữa.

✓ Publish / Subscribe (phát loa)



Thêm người nghe mới (consumer)
= chỉ cần "đăng ký lắng nghe",
BacktestWorker không đổi gì cả.

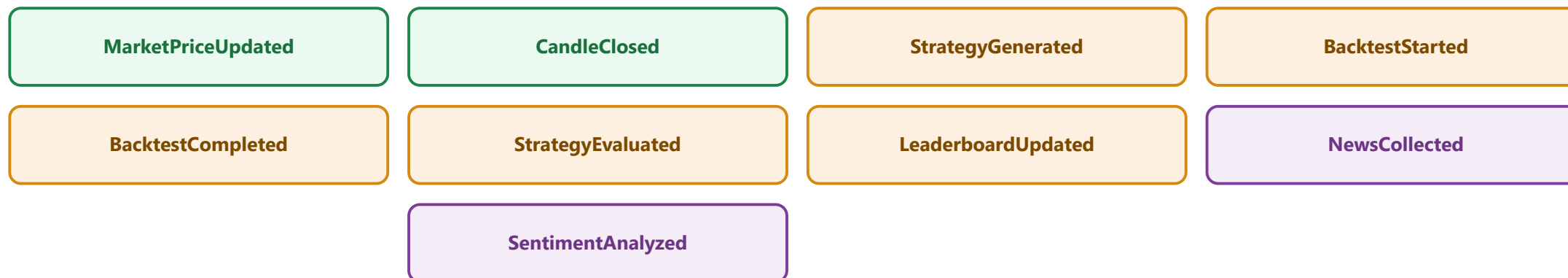
21. Event Catalog của Crypto Strategy Lab

9 sự kiện: MarketPriceUpdated, CandleClosed, StrategyGenerated, BacktestStarted/Completed, StrategyEvaluated, LeaderboardUpdated, NewsCollected, SentimentAnalyzed.

Với mỗi event, phải hỏi: owner? schema/version? key/order? duplicate? consumer failure? cần replay không? (*chi tiết ở slide sau*)

Event name dễ. Event semantics mới khó.

Event Catalog — mỗi "kiện hàng" cần nhãn chuẩn



Đặt tên sự kiện = dễ. Nhãn dán đầy đủ mới khó:



"Event name dễ. Event semantics mới khó."

22. Event Streaming và "Kappa thinking": market data là dòng sự kiện liên tục

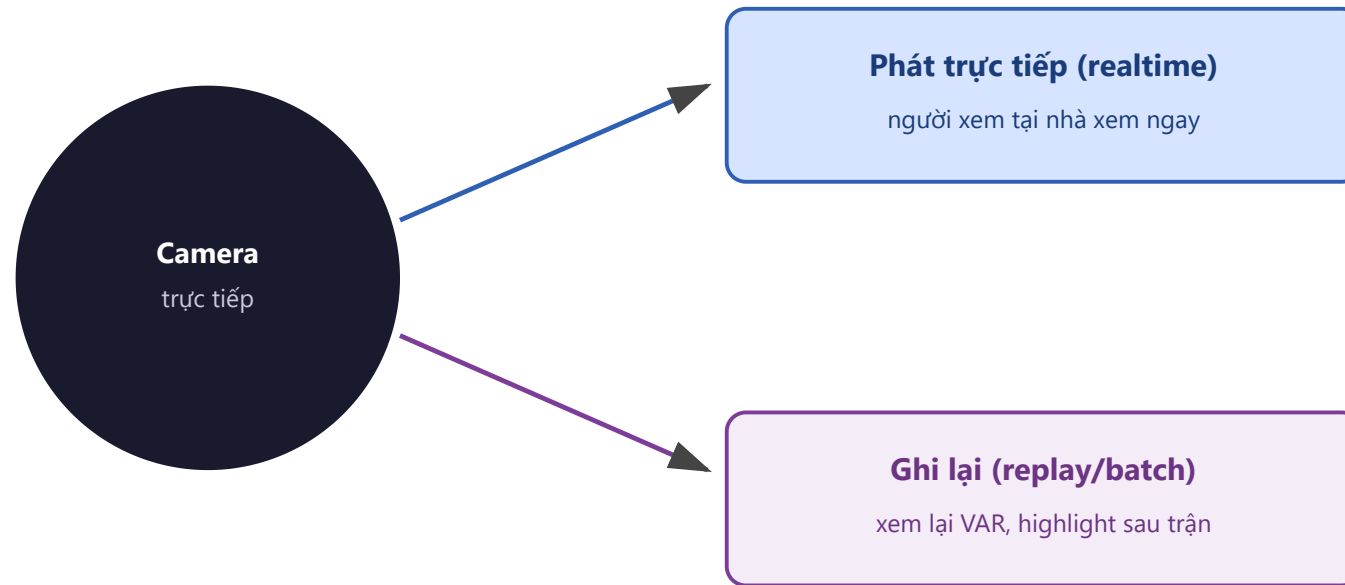
Binance WebSocket → Exchange Adapter → Normalized Market Events → Indicator/Strategy/UI/Storage

Kappa-style intuition: một luồng xử lý chính cho streaming, thay vì duy trì hai logic batch+speed riêng. Nhưng không phải project nào cũng cần Kafka/Kappa. (*minh họa ở slide sau*)

Syllabus: Event Streaming, Kappa Architecture

Nguồn: [R22], [R23], [W6]

Kappa thinking: một nguồn tín hiệu, hai cách dùng



Tương đương với market data:

Binance WebSocket → Exchange Adapter → Normalized Market Events

→ dùng cho cả Indicator/Strategy/UI (realtime) lẫn Storage (batch)

*"Use streaming because your problem is a stream —
not because Kafka is fashionable."*

23. Serverless có thể nằm ở đâu?

Candidate hợp lý

```
Scheduled News Fetch
  ↓
Serverless Function
  ↓
Normalize NewsItem
  ↓
Publish NewsCollected
```

Không phải lựa chọn mặc định cho

- long-running backtest worker (worker chạy dài)
- stateful high-throughput loop (vòng lặp lưu trạng thái, tải cao)
- low-latency connection giữ lâu (kết nối cần độ trễ thấp, giữ lâu)

Trade-off (đánh đổi)

- + scale-to-demand, ops nhẹ
- execution limits, cold start, state/external dependency complexity

Nhân viên thời vụ, so với nhân viên toàn thời gian

✓ Serverless — thời vụ

Scheduled News Fetch



Serverless Function



Normalize → Publish

Việc vài giây, mỗi giờ 1 lần

→ trả tiền theo lần gọi

(News Collector)

✗ Không hợp cho việc dài

long-running backtest worker

stateful high-throughput loop

low-latency connection giữ lâu

Việc chạy liên tục hàng giờ

→ cần nhân viên thường trực

(Backtest Worker)

Trade-off: + scale-to-demand, ops nhẹ

– execution limits, cold start

Thuê nhân viên thời vụ cho việc dài sẽ liên tục phải "đào tạo lại từ đầu" (cold start).

ACT 5 — "Có cần Microservices chưa?"

Trước hết: **Modular Monolith** là một đáp án hợp lệ

One deployable: [Market module | Strategy module | Experiment module | News module | API module]

Tách process/service khi có driver: độc lập scale · fault isolation · independent deployment · runtime/resource profile khác nhau

Syllabus: Microservice Architecture

Nguồn: [R13], [R14], [R15]

24. Docker: đóng gói execution environment (môi trường chạy)

```
image: crypto-backtest-worker:v1 → run → container #1, #2, #3
```

Giải quyết: runtime/dependency consistency · packaging · isolation · repeatable deployment

Không tự giải quyết: service boundaries · scaling policy · data consistency · observability

Syllabus: Containers (Docker)

Minh chứng: [W4]

25. Kubernetes: khi "thêm worker" phải trở thành thao tác có hệ thống

Backtest Queue → [W1, W2, W3, W4] replicas: 1 → 4

Phù hợp khi cần: scheduling · desired state · replicas · restart tự động · rolling deployment · autoscaling

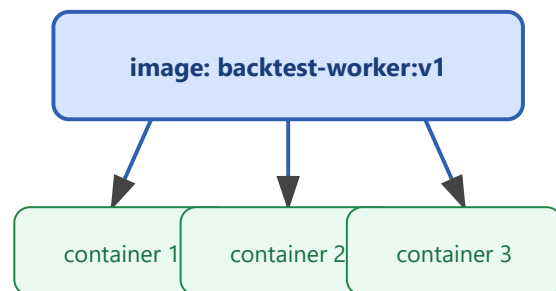
Không bắt buộc cho MVP. (*sơ đồ scale đầy đủ ở slide sau*)

Syllabus: Container Orchestration

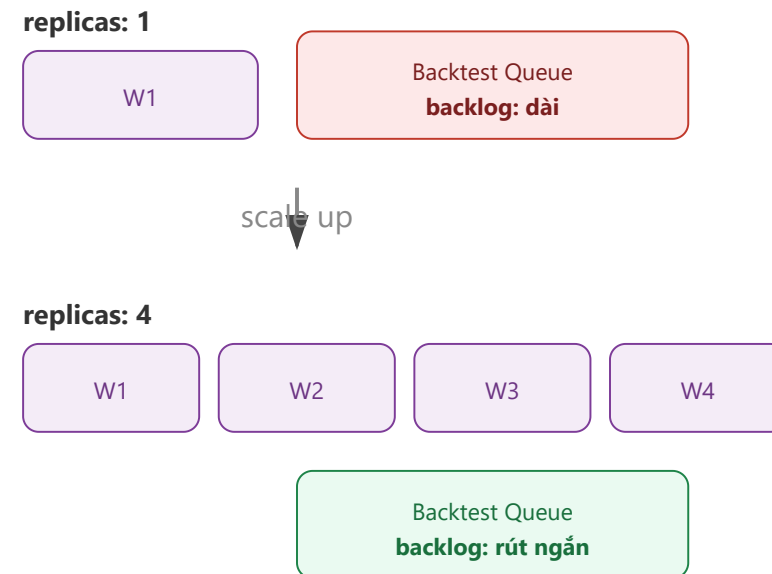
Minh chứng: [W5]

Docker đóng gói → Kubernetes nhân bản theo tải

1. Docker: 1 công thức, chạy ở đâu cũng giống



2. Kubernetes: tăng bản sao khi hàng đợi dài



Điều kiện: worker phải stateless và job phải idempotent — nếu không, thêm worker chỉ tạo ra nhiều lỗi hơn nhanh hơn.

26. Service Mesh: chỉ xuất hiện khi network trở thành "một hệ thống"

Với 6 service (Market, Strategy, Backtest, Ranking, News, Sentiment), mới có vấn đề cross-cutting: service-to-service traffic, retries/timeouts, mTLS, telemetry, traffic policy.

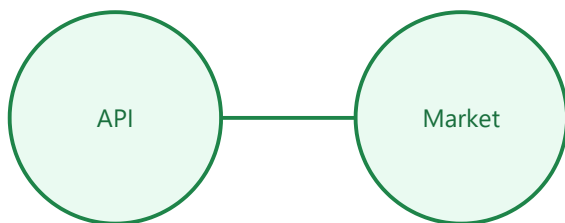
Nếu chỉ 2–3 process? Service Mesh có thể là **chi phí lớn hơn lợi ích**. (*minh họa ở slide sau*)

Syllabus: Service Mesh

Nguồn nền: [R14], [R13]

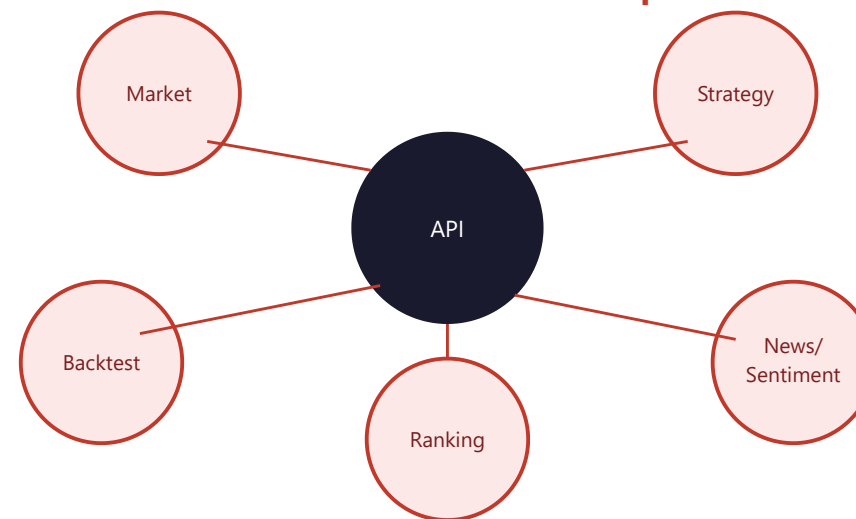
Service Mesh: khu phố 3 nhà chưa cần xây metro

✓ 2-3 service — đi bộ là đủ



Không cần Service Mesh —
chi phí lớn hơn lợi ích

⚠ Nhiều service — cần điều phối



retries · timeouts · mTLS ·
telemetry · traffic policy

→ đây mới cần Service Mesh

Xây hệ thống đèn giao thông cho khu phố 3 nhà là lãng phí — không phải "chuyên nghiệp hơn".

ACT 6 — "Top #1 này từ đâu ra?"

Kết quả demo: #1 MA+RSI+SR, Return 18.2%, MDD -6.1%, Trades 81

Câu hỏi kiểm tra: MA version nào? RSI parameters? dataset period? fee? code commit/model version? search config?

"...không có câu trả lời."

Đây là kiến trúc của một Experiment Platform (nền tảng thí nghiệm có thể kiểm chứng lại).

Project: [P §35–36]

Phòng thí nghiệm nghiêm túc: ghi lại MỌI điều kiện

✗ "Thí nghiệm thành công"

Return = 18.2%

MA version? RSI params?

dataset? model version?

"...không có câu trả lời"

✓ **Experiment Platform**

id, candidate spec

strategy versions, parameters

dataset / timeframe

execution config, metrics

timestamps, status

model version (nếu có sentiment)

→ mọi kết quả đều truy vết được

Nhà khoa học không chỉ ghi "thành công"
— họ ghi nhiệt độ, nồng độ, thiết bị, thời gian

để bất kỳ ai khác cũng có thể lặp lại thí nghiệm và ra cùng kết quả.

27. CQRS: write model và read model có thể khác nhau

Write side: RunBacktest → Experiment → Result/Metrics/Events

Read side: LeaderboardView (Rank | Strategy | Return | MDD | Trades)

Vì sao tách? Write tối ưu cho consistency/workflow; Read tối ưu cho tốc độ hiển thị.

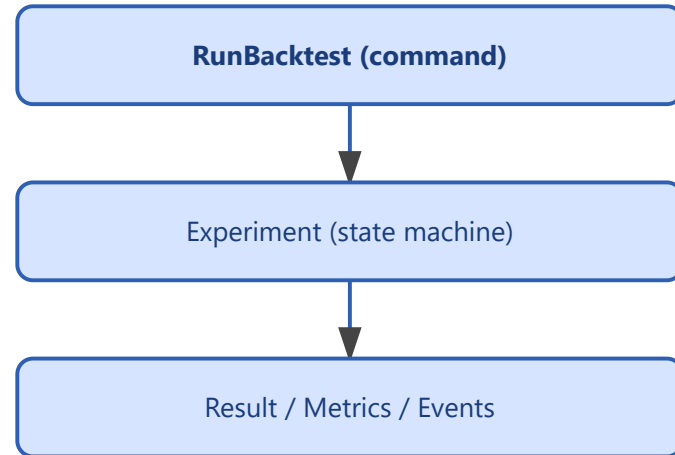
CQRS thêm complexity. Đừng dùng nếu CRUD đơn giản đã đủ. *(sơ đồ ở slide sau)*

Syllabus: CQRS (Command Query Responsibility Segregation)

Nguồn: [R4], [R10]

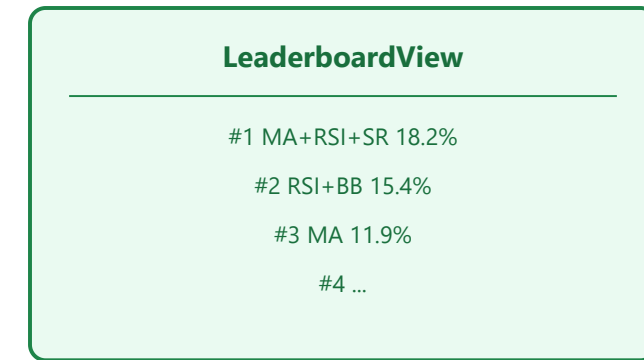
CQRS: bếp (ghi) và thực đơn (đọc) là hai mô hình khác nhau

Write side — "bếp"



Nhiều bước, phức tạp,
tối ưu cho tính đúng đắn.

Read side — "thực đơn"



Đơn giản, hiển thị nhanh,
tối ưu cho tốc độ đọc.

Chỉ tách khi CRUD đơn giản không còn đủ — CQRS là đánh đổi, không phải huy hiệu để khoe.

28. Event Sourcing: "lưu lịch sử thay đổi" khác "lưu trạng thái cuối"

```
State-only:      Experiment.status = COMPLETED
Event history: ExperimentCreated → CandidateAssigned → BacktestStarted
                → BacktestCompleted → StrategyEvaluated → LeaderboardPromoted
```

Lợi ích: audit · replay · temporal history · debugging — **Chi phí:** schema evolution · storage · overhead

Không bắt buộc cho đề án. (sơ đồ ở slide sau)

Syllabus: Event Sourcing

Nguồn: [R4], [W7]

Số dư hiện tại, so với sao kê đầy đủ lịch sử

State-only

Số dư hiện tại
5.000.000đ

Biết ĐANG có bao nhiêu,
không biết vì sao lại là con số đó.

Event history (sao kê)

Sao kê giao dịch

01/08 ExperimentCreated
01/08 CandidateAssigned
02/08 BacktestStarted
03/08 BacktestCompleted
03/08 StrategyEvaluated
04/08 LeaderboardPromoted

Biết CHÍNH XÁC điều gì xảy ra,
theo thứ tự nào — dựng lại được state bất kỳ lúc nào.

Đổi lại: chi phí lưu trữ và độ phức tạp cao hơn — chỉ dùng khi audit/replay thực sự cần thiết.

29. News + Sentiment: ML là một component, không phải "trung tâm vũ trụ"

```
News Provider → News Collector → Normalized NewsItem  
→ Sentiment Service (BERT → FinBERT → LLM) → SentimentResult → SentimentStrategy
```

Strategy Engine **không** nên biết model cụ thể. (sơ đồ ở slide sau)

Project: [P §27–30, §44]

Syllabus: Quality Attributes for AI Systems, MLOps

ML là một component — không phải "trung tâm vũ trụ"



Giống thuê công ty dịch thuật, không phải một phiên dịch cụ thể

Chỉ cần nhận bản dịch chuẩn — đổi phiên dịch viên (đổi model)
không ai ở phía nhận biết được, miễn định dạng không đổi.

Đổi provider không đổi model · đổi model không đổi collector — mỗi khối thay đổi độc lập.

30. MLOps: kết quả phải trả lời "model nào tạo ra nó?"

Một prediction record nên trace được: `newsId, sentiment, score, model{name, version}, inputVersion, createdAt`

Monitor ít nhất: model/version đang deploy · inference failures · latency · input/data issues · quality drift (*chi tiết ở slide sau*)

Syllabus: Topic 11 – MLOps

Nguồn: [R24]

Tem nhãn lô sản xuất — truy vết chính xác model nào gây ra kết quả

Hộp sữa

Ngày sản xuất · Dây chuyền · Lô hàng

Prediction record

model: SentimentModel v3

newsId: 8821 · sentiment: NEGATIVE · score: 0.91

model: { name: SentimentModel, version: v3 }

inputVersion / preprocessingVersion / createdAt

**Monitor: model/version deploy · inference failures ·
latency · input/data issues · quality drift**

→ có sự cố, truy đúng LÔ (version) — không nghi ngờ cả hệ thống

Nếu phát hiện lô sữa lỗi, nhà máy thu hồi đúng lô đó — không phải toàn bộ sản phẩm đã bán.

31. AI Agent: mở rộng Search Engine, không thay toàn bộ architecture

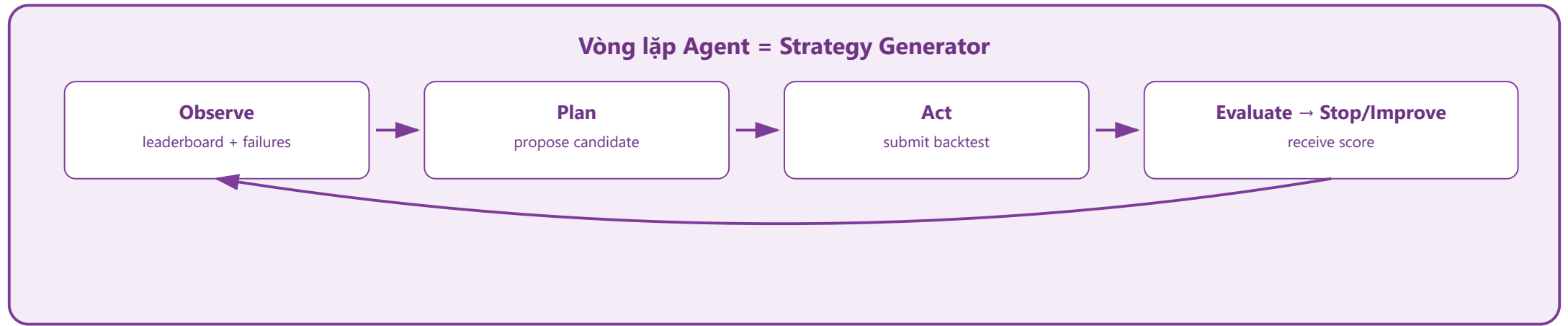
Agent = Strategy Generator: Observe → Plan → Act → Evaluate → Stop/Improve

Vẫn phải tuân theo contract: `StrategyGenerator.generate()` -> `CandidateStrategy` (sơ đồ ở slide sau)

Syllabus: AI Agent Frameworks & Design Patterns

Nguồn: [R25], [R26]

AI Agent: nhân viên mới, vẫn theo đúng biểu mẫu cũ



```
interface StrategyGenerator
    generate() -> CandidateStrategy
```

Backtester/Evaluator không biết candidate sinh bằng cách nào

Nhân viên mới (Agent) có thể tự học, tự nghĩ — nhưng vẫn nộp đúng biểu mẫu cũ.

ACT 7 — Thiết kế kiến trúc có phương pháp

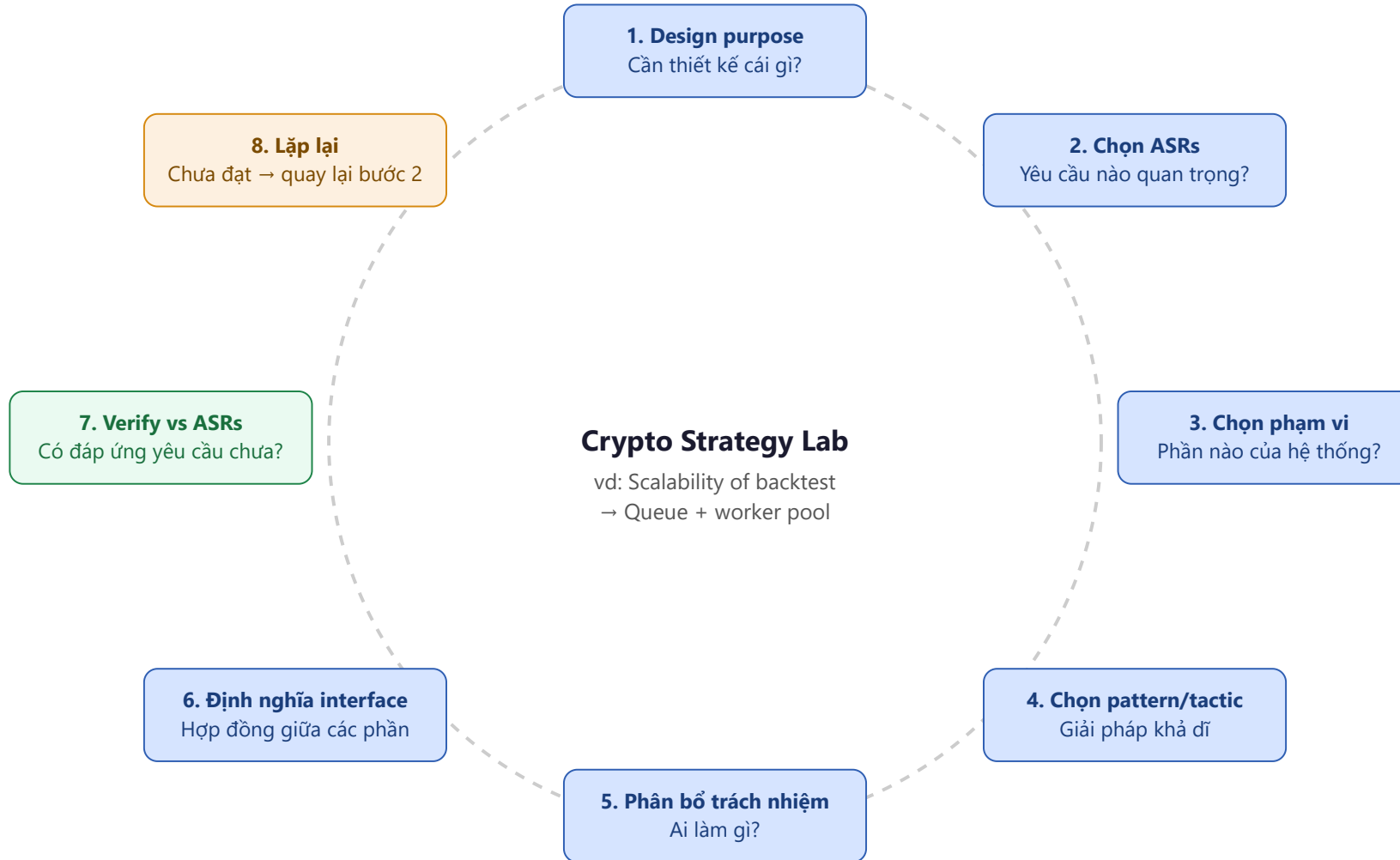
ADD: Driver → Decision → Decomposition (chia nhỏ dần, 8 bước — xem slide sau)

Áp dụng: Scalability of backtest → Queue + worker pool → define BacktestJob contract → đo throughput/failure

Syllabus: Attribute-Driven Design

Nguồn: [R6], [W1]

ADD: 8 bước, đi vòng tròn cho đến khi đạt yêu cầu



Bước 7 "verify" không đạt thì quay lại bước 2 — đây là vòng lặp, không phải đường thẳng một lần.

32. ATAM: Architecture không được chấm bằng "nhìn đẹp"

Ta đưa scenario vào "đập" kiến trúc

Scenario A

Thêm MACD → sửa mấy component?

Scenario B

Binance disconnect → recover thế nào?

Scenario C

100 → 100.000 backtests → bottleneck ở đâu?

Scenario D

News Service down → Market chart còn hoạt động?

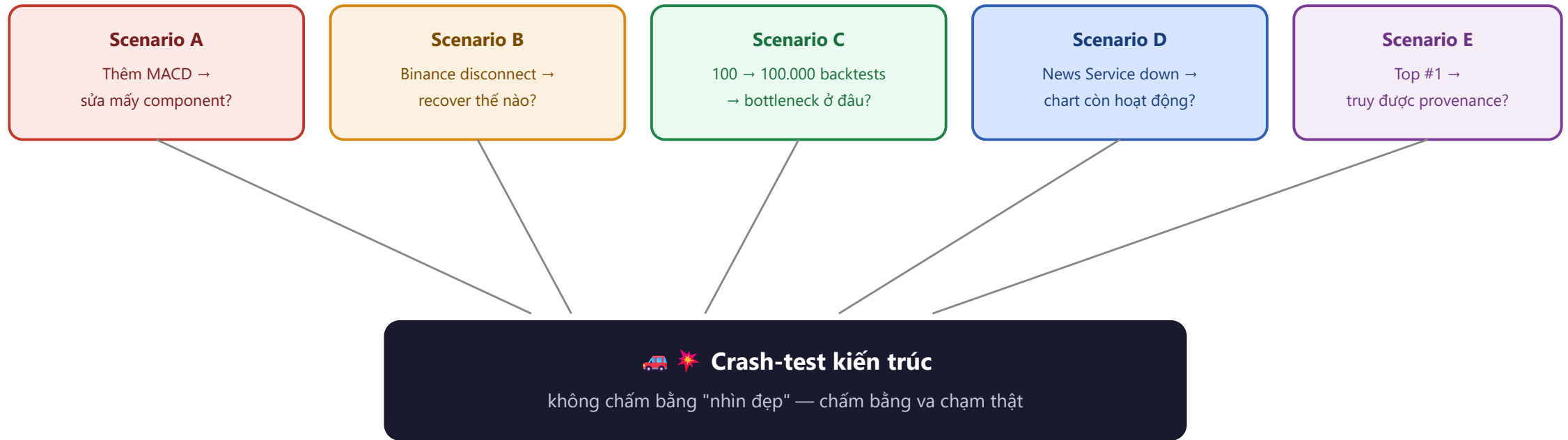
Scenario E

Top #1 → truy được provenance?

Syllabus: Architecture Evaluation / ATAM

Nguồn: [R1], [W8]

ATAM: crash-test 5 kịch bản vào kiến trúc



Mỗi lần "đâm" thành công một quyết định, luôn có một cái giá đi kèm — bảng trade-off ở slide sau.

33. Trade-off Matrix — không có "free lunch" (không có gì miễn phí)

Decision	Lợi ích	Giá phải trả
Plugin architecture	thêm strategy dễ	contract/versioning
Async queue	scale, retry	eventual consistency
Microservices	độc lập deploy/scale	network + ops complexity
Event-driven	loose coupling	tracing/order/duplicate
CQRS	read model linh hoạt	sync/projection complexity
Event Sourcing	audit/replay	schema/replay overhead
Kubernetes	orchestration/replicas	operational complexity
AI Agent search	flexible exploration	cost, nondeterminism, eval

Kiến trúc tốt = **trade-off** phù hợp với **driver**, không phải nhiều công nghệ.

34. Architecture Proof #1 — Extensibility Test (kiểm tra khả năng mở rộng)

Yêu cầu ngay khi demo: "Thêm MACD."

Thiết kế tốt: + MACDStrategy, + StrategyRegistry.register(MACDStrategy) → 2 dòng

Coupling cao: Controller, Backtester, UI, Database, CombinationEngine, Evaluator → 6 nơi sửa

Thay đổi thật là unit test của architecture.

Project: [P §41]

35. Architecture Proof #2 — Replaceability Test (kiểm tra khả năng thay thế)

Hiện tại: RandomStrategyGenerator

Yêu cầu mới: DomainGuidedStrategyGenerator

Contract (không đổi): `interface StrategyGenerator { generate() -> CandidateStrategy }`

Downstream (Backtester, Evaluator, Leaderboard) **không cần biết** candidate sinh bằng cách nào.

Project: [P §42]

36. Architecture Proof #3 — Scalability & Failure Test (kiểm tra khả năng mở rộng và chịu lỗi)

Test 1 — Scale: Workers 1 → 3. Hỏi: throughput đổi? queue backlog? DB contention? duplicate?

Test 2 — Failure: News Service = DOWN. Kỳ vọng: Realtime Chart ✓, Strategy ✓, Backtest ✓ — chỉ News/Sentiment degraded.

Test 3 — Realtime recovery: Binance disconnect → reconnect → gap recovery. *(Sơ đồ đầy đủ ở slide ảnh kế tiếp.)*

Project: [P §32, §40, §43]

3 bài kiểm tra "Architecture Proof" khi demo trực tiếp

1. Extensibility — "Thêm MACD"

Thiết kế tốt: 2 dòng thay đổi
+ MACDStrategy, register()

Coupling cao: 6 nơi phải sửa

Controller, Backtester, UI, DB, CombinationEngine, Evaluator

2. Replaceability — "Đổi Random → Genetic Search"

RandomGenerator → GeneticGenerator

interface StrategyGenerator
generate() -> CandidateStrategy

Backtester · Evaluator · Leaderboard
— không đổi một dòng nào

3. Scalability & Failure — "Tăng worker" + "Tắt News Service"

Workers: 1 → 3
đo throughput/backlog

News Service = DOWN
Chart/Backtest có sống sót?

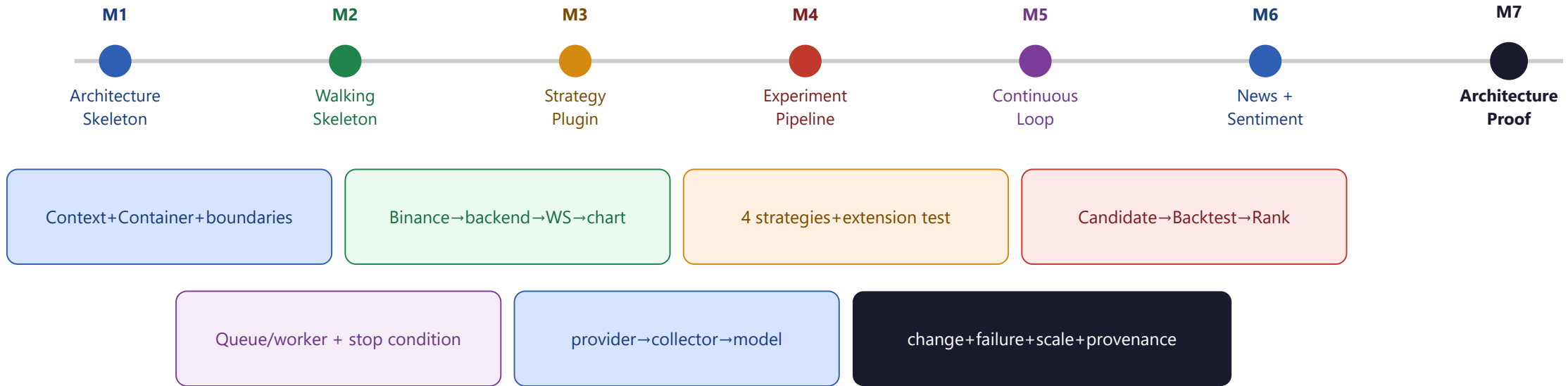
Kỳ vọng: Realtime Chart ✓ Strategy ✓ Backtest ✓
News/Sentiment: degraded — KHÔNG kéo sập cả hệ thống
+ Binance disconnect → reconnect → gap recovery

"Một tài liệu kiến trúc chỉ là lời tuyên bố. Một thay đổi thật diễn ra trực tiếp mới là bằng chứng."

37. 7 Milestone để làm đồ án không bị "vỡ trận"

- **M1 Architecture Skeleton** — Context + Container + boundaries + interfaces
- **M2 Walking Skeleton** — Binance → backend → WebSocket → chart (chạy được end-to-end)
- **M3 Strategy Plugin** — 4 strategies + extension test
- **M4 Experiment Pipeline** — Candidate → Backtest → Evaluate → Rank
- **M5 Continuous Loop** — Queue/worker nếu cần + stop condition
- **M6 News + Sentiment** — provider → collector → model → result
- **M7 Architecture Proof** — change + failure + scale + provenance

7 Milestone: đổ móng cả căn nhà trước, hoàn thiện dần



Thợ giỏi đổ móng và dựng khung cả căn nhà trước — không xây trọn từng phòng rồi mới ráp lại.

38. Demo cuối kỳ = phần kết câu chuyện

BTCUSDT (5m|15m|1h|4h) → Select MA/RSI/BB/SR → START SEARCH
→ Candidates tested: 125 → Leaderboard updates → Click Top #1
→ Trades+Return+MDD+signals → News+Sentiment → Add SentimentStrategy
→ Run search again

Sau đó mới "đập" architecture bằng change scenario

Project: [P §46]

39. Checklist: sinh viên phải trả lời được 10 câu này

1. Architectural drivers là gì?
2. C4 Context và Container của nhóm?
3. Boundary của Market / Strategy / Experiment / News?
4. Thêm strategy mới sửa ở đâu?
5. Đổi search algorithm sửa ở đâu?
6. Provider mới có làm frontend đổi?
7. 100.000 backtests scale thế nào?
8. Service lỗi có lan failure không?
9. Duplicate/retry/event order xử lý thế nào?
10. Leaderboard result truy được provenance thế nào?

Không trả lời được → kiến trúc vẫn đang là "hộp và mũi tên".

40. Mental model cuối cùng

Làm kiến trúc theo thứ tự này

1. Understand problem → 2. ASRs/Quality Attr. → 3. Write scenarios
→ 4. C4 views → 5. Boundaries & contracts → 6. Patterns/tactics
→ 7. Walking skeleton → 8. Measure/observe/fail → 9. Trade-offs
→ 10. Record ADRs + evidence

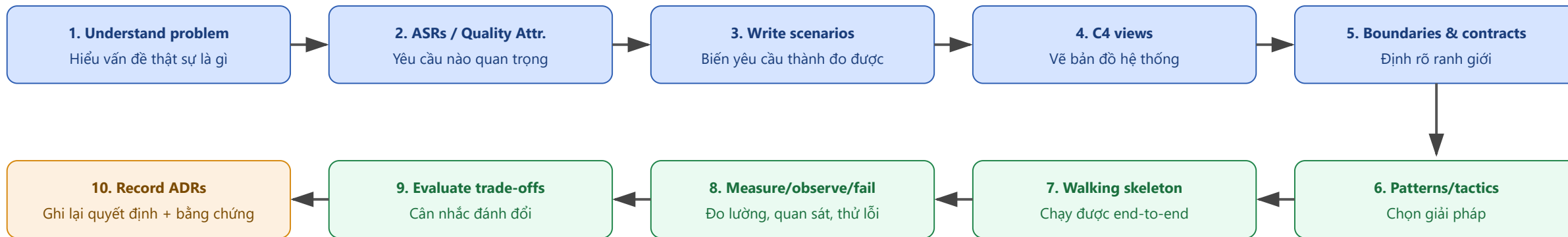
(Sơ đồ đầy đủ ở slide ảnh kế tiếp.)

Câu chốt

Đừng hỏi "Dùng công nghệ gì?" trước.

Hãy hỏi: "Điều gì sẽ thay đổi, điều gì có thể hỏng, và kiến trúc của ta chứng minh được gì?"

10 bước làm kiến trúc — từ hiểu vấn đề đến có bằng chứng



"Đừng hỏi dùng công nghệ gì trước."
Hãy hỏi: điều gì sẽ thay đổi, điều gì có thể hỏng,
và kiến trúc của ta chứng minh được gì.

Team 404 — Profit Not Found · Crypto Strategy Lab

PHỤ LỤC A — Đối chiếu Seminar ↔ Syllabus

Syllabus Topic	Nơi xuất hiện trong câu chuyện
1. Software Architecture Concepts	God Service → Architecture under change
2. Quality Attributes / ASRs / AI QA	change requests, scenarios, ML subsystem
3. 4+1 / C4 / Views / UML	Context, Container, Component, Dynamic view
4. Styles / Patterns / Reconstruction / ADD / ATAM	Plugin, Adapter, ADD, ATAM-lite
5. Microservices / Docker / Kubernetes / Service Mesh	scaling/deployment act
6. SPA / MFE / JAMstack / DDD / Clean / Transactions	UI choices + boundaries
8. CQRS / Event Sourcing	experiment/leaderboard/provenance
9. Brokers / Streaming / EDA / Serverless / Kappa	strategy factory pipeline
10. AI Agents	AgentStrategyGenerator extension
11. MLOps	sentiment model lifecycle/versioning

PHỤ LỤC B — Gợi ý ADR (Architecture Decision Record)

ADR-001 Why MarketDataProvider + Adapter?
ADR-002 Why WebSocket for realtime UI?
ADR-003 Why Strategy Plugin/Registry?
ADR-004 Why separate Backtester and Evaluator?
ADR-005 Why queue/worker (or why NOT)?
ADR-006 Why modular monolith vs microservices?
ADR-007 How experiment/version provenance is stored?
ADR-008 Why separate News Collector and Sentiment Service?
ADR-009 Why CQRS/Event Sourcing is used – or deliberately not used?
ADR-010 Stop conditions and observability of Strategy Loop

Format ngắn: Context · Decision · Alternatives · Consequences · Evidence

Project: [P §45]

PHỤ LỤC C — Rubric đánh giá kiến trúc (gợi ý)

Tiêu chí	Câu hỏi kiểm chứng
Modifiability (dễ sửa)	thêm MACD sửa bao nhiêu component?
Replaceability (dễ thay thế)	Random → Domain-guided có ảnh hưởng Backtester?
Scalability (mở rộng)	worker count tăng có cần sửa code core?
Reliability (tin cậy)	kill News/Binance connection hệ thống degrade ra sao?
Observability (quan sát được)	có biết queue depth, job latency, failure count?
Reproducibility (tái lập được)	Top-K có link về exact experiment config/version?
Documentation (tài liệu)	Context/Container/Dynamic view nhất quán với code?
Trade-off reasoning (lý giải đánh đổi)	mỗi công nghệ có driver và consequence?

PHỤ LỤC D — Ký hiệu nguồn trong seminar

- **[S]** Syllabus - Software Architecture (4).docx
 - §5 Teaching Plan
 - §7 Resources / References
- **[P]** Crypto Strategy Lab – Đồ án cuối kỳ(1).pdf
 - dùng số mục § đúng theo project spec
- **[R#]** sách nằm trong mục **References** của syllabus (R1–R26)
- **[W#]** nguồn official/primary dùng để kiểm chứng thêm (W1–W8)

Nội dung [P]/[S] được giữ theo framing của tài liệu môn học.

Các mở rộng như "ATAM-lite", milestone, architecture proof là cách tổ chức giảng dạy được suy ra từ tài liệu và literature, không phải yêu cầu mới của đề nếu đề không ghi.

Bản dễ hiểu này bổ sung thêm ví dụ minh họa và sơ đồ SVG so với bản gốc; các ví dụ đời thường (nhà hàng, ATM, sao kê ngân hàng...) là minh họa sơ phạm do người biên soạn thêm vào, không phải trích dẫn từ [S]/[P]/[R#]/[W#].

PHỤ LỤC E — References từ syllabus (1/3)

[R1] Len Bass, Paul Clements, Rick Kazman (2021).

Software Architecture in Practice, 4th ed. Addison-Wesley.

[R2] Robert C. Martin (2017).

Clean Architecture: A Craftsman's Guide to Software Structure and Design. Pearson.

[R3] Vlad Khononov (2021).

Learning Domain-Driven Design. O'Reilly.

[R4] Ethan Garofolo (2020).

Practical Microservices: Build Event-Driven Architectures with Event Sourcing and CQRS.

Pragmatic Bookshelf.

[R5] Paul Clements et al. (2010).

Documenting Software Architectures: Views and Beyond. Pearson.

[R6] Humberto Cervantes, Rick Kazman (2016).

Designing Software Architectures: A Practical Approach. Addison-Wesley Professional.

[R7] Nick Rozanski, Eoin Woods (2012).

PHỤ LỤC F — References từ syllabus (2/3)

[R9] Erich Gamma et al. (1994).

Design Patterns: Elements of Reusable Object-Oriented Software. Addison-Wesley.

[R10] Martin Fowler et al. (2002).

Patterns of Enterprise Application Architecture. Addison-Wesley.

[R11] Philip A. Bernstein, Eric Newcomer (2009).

Principles of Transaction Processing, 2nd ed. Morgan Kaufmann.

[R12] Emmitt Scott (2015).

SPA Design and Architecture. Manning.

[R13] Chris Richardson (2019).

Microservices Patterns: With Examples in Java. Manning.

[R14] Sam Newman (2021).

Building Microservices: Designing Fine-Grained Systems. O'Reilly.

[R15] Sam Newman (2019).

Monolith to Microservices. O'Reilly.

PHỤ LỤC G — References từ syllabus (3/3)

[R18] Mathias Biilmann, Phil Hawksworth (2019).

Modern Web Development on the JAMstack. O'Reilly.

[R19] Raymond Camden, Brian Rinaldi (2022).

The Jamstack Book. Manning.

[R20] Eric Evans (2003).

Domain-Driven Design: Tackling Complexity in the Heart of Software. Addison-Wesley.

[R21] Vaughn Vernon (2013).

Implementing Domain-Driven Design. Addison-Wesley Professional.

[R22] Martin Kleppmann (2016).

Making Sense of Stream Processing. O'Reilly.

[R23] Nathan Marz, James Warren (2015).

Big Data: Principles and Best Practices of Scalable Realtime Data Systems. Manning.

[R24] Christian Kästner (2025).

Machine Learning in Production: From Models to Products. MIT Press.

PHỤ LỤC H — Nguồn chính thức dùng để kiểm chứng (official/primary sources)

[W1] CMU Software Engineering Institute

Attribute-Driven Design Method Collection (sei.cmu.edu) — ADD dựa trên ASRs/quality attribute requirements và recursive decomposition. Đã kiểm chứng qua web search khi biên soạn bản dễ hiểu này.

[W2] C4 Model official site — Simon Brown (c4model.com)

C4 static structure: System Context → Container → Component → Code; Context + Container thường đủ cho nhiều team. Đã kiểm chứng.

[W3] Apache Kafka official documentation (kafka.apache.org)

Event streaming, producers, consumers, topics, durable event streams và decoupling producer/consumer.

[W4] Docker official documentation (docs.docker.com)

Container là runnable instance của image; image/container hỗ trợ repeatable packaging/execution.

[W5] Kubernetes official documentation (kubernetes.io)

PHỤ LỤC I — Câu hỏi tương tác dự phòng

1. "Kafka có bắt buộc không?"

Không. Hãy chứng minh vì sao queue/event broker cần cho scale/coupling của nhóm.

2. "Microservices có được điểm cao hơn monolith?"

Không mặc định. Modular monolith có boundary tốt có thể tốt hơn distributed monolith.

3. "Có cần CQRS + Event Sourcing?"

Không. Chỉ dùng khi read/write shape, audit/replay hoặc domain driver đủ mạnh.

4. "Strategy có được query DB?"

Nên tránh để domain strategy phụ thuộc trực tiếp infrastructure; truyền context/port phù hợp.

5. "ML model tốt nhất có quyết định điểm?"

Đề nhấn mạnh architecture. ML là component cần boundary, versioning và evaluation phù hợp.

PHỤ LỤC J — Một câu để nhớ từng chương

- **Quality Attributes:** "Tốt theo nghĩa nào, trong scenario nào?"
- **C4:** "Đang zoom ở mức nào?"
- **DDD:** "Boundary ngữ nghĩa nằm ở đâu?"
- **Clean Architecture:** "Business policy có đang phụ thuộc infrastructure?"
- **Patterns:** "Variability/force nào khiến pattern này tồn tại?"
- **Event-driven:** "Sự kiện nào đã xảy ra, ai thực sự cần biết?"
- **Microservices:** "Tại sao phải deploy/scale độc lập?"
- **CQRS:** "Write model và read model có thật sự cần khác?"
- **Event Sourcing:** "Có cần lịch sử/replay đủ mạnh để trả complexity?"
- **MLOps:** "Prediction này do model/data/version nào tạo?"
- **ATAM:** "Scenario nào có thể làm architecture thất bại?"

END